

Axona Services Guide

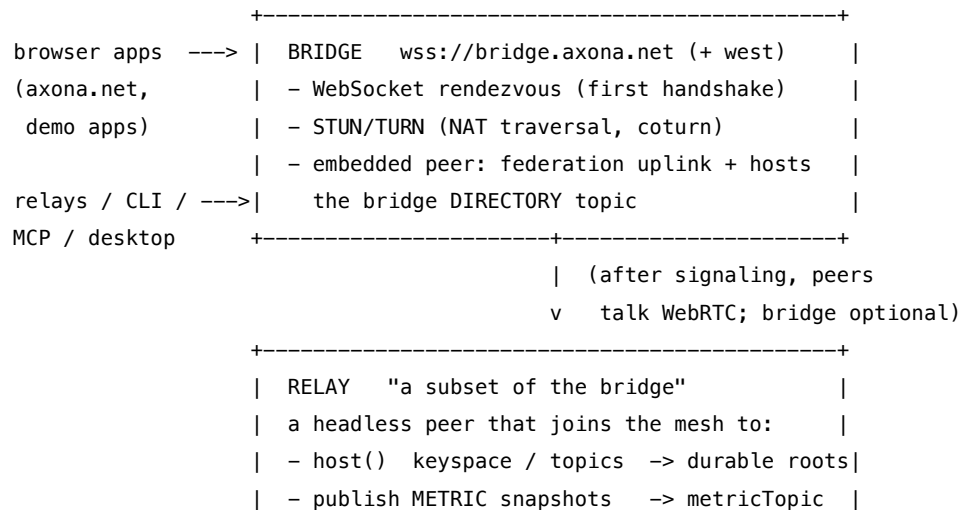
(kernel 4.11.2 · testnet — versioned with the protocol it describes)

The other programmer-guide documents teach the **library** — how to build your own peer with @axona/protocol and speak pub/sub. This guide covers the **services**: the ready-made programs you *run* (or rent) rather than write — the signaling **bridge**, the **relay** and its four front-ends (console, CLI, MCP server, desktop app), the **directory/federation** that ties bridges together, and the **PoW collector**. If you are operating an Axona deployment, wiring an agent into the network, or just want a topic to stay alive when no browser tab is open, this is the document for you.

- **Protocol kernel**: @axona/protocol (v4.11.2)
- **Wire version**: 4.0 (WIRE_VERSION); kernel version 4.11.2 (KERNEL_VERSION)
- **Live network**: the 4.x line runs on **testnet** — wss://testnet.axona.net. The **production** federated pair (wss://bridge.axona.net east + wss://bridge-west.axona.net west) is still on the 3.x line; the two don't interoperate (the wire major partitions them). Run 4.x services against testnet — point relays/MCP at it explicitly (their code default still targets prod/3.x; see §3).
- **Companion docs**:
 - Quick Start · Programmer Guide · API Reference — the library.
 - AI Grounding — building with an AI assistant.
 - Architecture — how the bridge + transport work under the hood.

Library vs. services. A *peer* is the unit the library gives you: an identity + a transport + a pub/sub manager, running inside a browser tab, a Node process, or an agent. The library is enough to build a complete app. The **services** exist because some jobs outlive a single tab: brokering the first WebRTC handshake between two strangers (the **bridge**), keeping a topic's messages available when every author has closed their laptop (a **relay** that host()s the topic), and letting non-browser callers — shells, agents, GUIs — publish and subscribe (the **CLI** / **MCP** / **desktop** front-ends). None of them is privileged: every service is just a peer that has volunteered to stay online and take on a role.

1. The service map



```
| front-ends over one core (src/ops.js):      |
| console/TUI . CLI . MCP server . desktop app |
+-----+
```

Service	Package / entry	What it is	Runs where
Bridge	axona-bridge (src/server.js)	Signaling rendezvous + STUN/TURN + a federating, directory-hosting peer	A server (Docker stack)
Relay — console/TUI	axona-relay (src/index.js, bin axona-relay)	Headless node that hosts topics + publishes metrics; live dashboard	A box you keep online
Relay — CLI	axona-cli (src/cli.js)	One-shot pub / sub / pull from a shell or script	Anywhere with Node
Relay — MCP server	axona-mcp (src/mcp.js)	A persistent peer with a durable identity, exposed as native agent tools (publish/subscribe/host) over MCP/stdio	Spawned by the agent host
Relay — desktop app	axona-relay/desktop (Electron)	A GUI wrapper around the relay	A laptop/desktop
PoW collector	axona-relay/pow-collector.js (npm run collect)	Subscribes to the PoW-bench results topic and archives them	A box you keep online
Directory / federation	kernel bridgeDirectory.js	A <i>behavior</i> , not a process: bridges advertise, clients fail over	Inside bridges + clients

The same @axona/protocol kernel runs in every box above. There is no special “server build” — a bridge is a peer that also terminates WebSockets and TURN; a relay is a peer that also `host()`s and publishes metrics.

2. The Bridge

What it is

A **bridge** is the only piece of fixed infrastructure Axona needs, and even it is needed only at the **start** of a connection. Two browsers that have never met cannot exchange WebRTC offer/answer SDP on their own — they need a mutually reachable rendezvous. The bridge is that rendezvous. It does three jobs:

1. **WebSocket signaling** — relays the offer/answer/ICE handshake between two peers until their direct WebRTC data channel is open. After that the bridge is **out of the data path**; peers talk peer-to-peer over the mesh.
2. **STUN/TURN** — a bundled **coturn** gives peers their public reflexive address (STUN) and, for the ~10–15% of NAT pairs that can't connect directly, relays the media (TURN). The bridge mints short-lived TURN credentials in its welcome frame.
3. **An embedded peer** — the bridge process also *joins its own network* as a node. That embedded peer **federates** (an outbound uplink to a sibling bridge, §5) and **hosts the bridge directory topic** so clients can discover other bridges and fail over.

A bridge is a convenience, not an authority. It cannot read message payloads (they're end-to-end between mesh peers), cannot forge a publish (every envelope is author-signed), and any number of independent bridges can coexist — clients rank and fail over between them.

Running one

Production bridges run as a **Docker Compose stack** — **bridge** + **Caddy** (automatic TLS) + **coturn** (TURN) — from the `axona-bridge` repo. The one-command path for a fresh Ubuntu/Debian host:

```
curl -fsSL https://raw.githubusercontent.com/axona-net/axona-bridge/main/deploy/install.sh \
| sudo BRIDGE_DOMAIN=bridge.example.net LETSENCRYPT_EMAIL=you@example.net bash
```

or, with the repo checked out:

```
cd /opt/axona-bridge
docker compose build      # builds first; the old container keeps serving
docker compose up -d      # fast swap; Caddy keeps its certificate
curl https://bridge.example.net/healthz # →
↪ {"status":"ok","version": "...", "kernelVersion": "4.11.2"}
```

Full provisioning options (the installer, the Docker bundle, and a manual walkthrough) are in `axona-bridge/deploy/INSTALL.md`.

Configuration

Bridge config lives in `/etc/axona-bridge.env` (systemd) or the compose `.env`:

Var	Meaning
DOMAIN / PUBLIC_IP	Public hostname + IP (TLS + TURN advertisement)
TURN_AUTH_SECRET	Shared secret with coturn's <code>static-auth-secret</code> ; the bridge mints creds from it

Var	Meaning
BRIDGE_PUBLIC_URL	The <code>wss://</code> URL this bridge advertises in the directory (e.g. <code>wss://bridge-west.axona.net</code>)
BRIDGE_DIRECTORY	on (advertise + host the directory) or off (isolated fleet — testnet uses off)
BRIDGE_LAT / BRIDGE_LNG / REGION_LABEL	The bridge's geo anchor + a human label for its directory entry
MIN_PEER_VERSION	Lowest wire version admitted; older peers get <code>UPGRADE_REQUIRED</code> (WS close 4426)
HOST	Bind address — production sets <code>127.0.0.1</code> so the reverse proxy is the only ingress
HEALTHZ_TOKEN	Gates the full <code>/healthz</code> body + <code>/diag</code> ; unauthenticated <code>/healthz</code> returns only <code>{status,version,kernelVersion}</code>

Health & introspection

- `GET /healthz` — unauthenticated, returns `{status, version, kernelVersion}`.
- `GET /healthz` with header `X-Healthz-Token: <token>` — the full body: synaptome size, directory `{enabled,url}`, uplink status, peer counts.
- `GET /diag` (token-gated) — deeper diagnostics.

3. The Relay

What it is

A **relay** is a **subset of the bridge**: the embedded-peer half, without the WebSocket/TURN server. It is a headless node you keep online so the network has durable, well-placed members. Two jobs:

1. **Hosting** — `host()` makes the relay a **root** for a keyspace region (or for named topics) *without subscribing*, so it stores-and-serves a topic's messages even when it doesn't consume them. This is what keeps a topic's backlog answerable after every author closes their tab. By default a relay hosts its region's keyspace (`0x89` for `useast`). Under the 4.x **cohort** model a topic is replicated across the K-closest nodes, so durability no longer rests on one relay — several co-hosting relays converge on the same history via anti-entropy, and any of them can answer a read.
2. **Metrics** — every ~20 s the relay walks its rooted topics and publishes a signed snapshot (`{current_count, seq, subscribers, bytes}`) to the derived `metricTopic(T)`, so clients can `sub()` for live counts + a rolling ~48 h trend instead of polling. As of v4.3.0 **both open and owned** topics publish metrics (owned-topic activity counts are public too); `seq` is the dense message counter (total events ever, kills included). Because every co-hosting root publishes its own

snapshot, the reader-side `peer.metrics()` aggregates across the cohort (sums subscribers, maxes the rest — see the API Reference).

Running one

```
cd axona-relay
npm install
npm start      # live dashboard (interactive TTY)
npm run probe  # plain timestamped log lines (RELAY_TUI=0) — for piping / services
```

Run it again in another terminal for a second node — the **first** instance claims the persistent identity (stable `nodeId`, `identity.<region>.json`); each **additional** instance mints a fresh ephemeral identity in the same region. So you get one well-known node plus as many throwaway nodes as you want.

Configuration (env)

Var	Default	Meaning
RELAY_NETWORK	prod	Bootstrap network: prod (<code>bridge.axona.net</code> , 3.x) or testnet (<code>testnet.axona.net</code> , 4.x). A v4.11.2 relay must use testnet — the code default prod is the 3.x line and rejects a 4.x peer at the handshake.
BRIDGE_URL	—	Explicit bridge URL; overrides RELAY_NETWORK
RELAY_REGION	—	auto (detect), a region name (useast), or code (0x89) — sets the <code>nodeId</code> geo prefix
RELAY_LAT / RELAY_LNG	37.77/-122.42	Geo prefix by coordinate (if RELAY_REGION unset). Default = SF (uswest)
RELAY_IDENTITY_PATH	./identity.<region>.json	Persisted keypair (stable <code>nodeId</code>)
RELAY_HOST_KEYSPACE	1	Host the region's whole keyspace (set 0 to host only RELAY_TOPICS)
RELAY_TOPICS	—	Comma-separated topic names to host explicitly
RELAY_METRICS	1	Publish metric snapshots (0 to disable)
RELAY_METRICS_INTERVAL_MS	~20000	Metric publish cadence (~20 s)

Var	Default	Meaning
RELAY_TUI	auto	1 force dashboard, 0 force plain log

Bridge precedence: `BRIDGE_URL` › `RELAY_NETWORK` › prod. Region precedence: `RELAY_REGION` › `RELAY_LAT/RELAY_LNG` › default SF.

Run as a service (systemd)

```
# /etc/systemd/system/axona-relay.service
[Service]
Environment=RELAY_REGION=useast RELAY_TUI=0
ExecStart=/usr/bin/node /opt/axona-relay/src/index.js
Restart=always
```

Operator note — process count. Each relay forks **one worker child**, so a healthy N -node fleet is $2 \times N$ node `src/index.js` processes (a parent + worker per node). Judge fleet health by the distinct **bridge connections** in the logs, not the raw process count. If the **desktop app** (below) is also running, it carries *its own* embedded relay that respawns if killed — expect one extra.

4. Relay front-ends

The relay's core pub/sub lives in `src/ops.js` (publish / subscribe / pull). The **CLI** uses it directly — one throwaway peer per call. The **MCP server** wraps it in a *persistent* peer (`src/mcp-session.js`) so an agent can be a standing participant rather than a series of one-shot connections. The **desktop app** is the GUI relay.

4a. CLI (`axona-cli`)

One-shot pub/sub/pull from a shell or script — no long-lived process:

```
npm run pub -- --topic us-east/hello --message "hi"           # prints { ok, msgId, ... }
npm run sub -- --topic us-east/hello --seconds 20            # JSON lines for N seconds
npm run pull -- --topic us-east/hello                        # the single latest message
```

Good for cron jobs, CI checks, shell glue, and quick manual probes.

4b. MCP server (`axona-mcp`)

`src/mcp.js` is a **Model Context Protocol** server that exposes Axona as **native agent tools**, so Claude Code (and any MCP-speaking agent) gets first-class `axona_*` tools instead of shelling out. It speaks JSON-RPC over **stdio** (stdout = protocol, stderr = logs).

It holds ONE persistent peer (`src/mcp-session.js`), opened lazily on the first call and kept connected — so the agent is a continuous participant, not a series of one-shot connections.

That peer signs with a **durable identity** (both the node identity and the author keypair persist to `~/.axona/claude-mcp-identity.json`), so the agent keeps the **same Author ID and nodeId across restarts** — a stable on-network identity, not a fresh persona each call.

Tool	Args	Purpose
<code>axona_publish</code>	<code>topic, message, region?</code>	Publish, signed by the stable author
<code>axona_pull</code>	<code>topic, region?</code>	Fetch only the latest message
<code>axona_watch</code>	<code>topic, region?, since? (all latest live)</code>	Open a standing subscription ; arrivals buffer server-side
<code>axona_poll</code>	<code>topic?, peek?, max?, wait?, timeoutSec?</code>	Drain the buffer (the agent’s “inbox”); <code>wait:true</code> long-polls until a message lands
<code>axona_unwatch</code>	<code>topic, region?</code>	Stop a watch, discard its buffer
<code>axona_host</code>	<code>topic, region?</code>	Root the topic on the agent’s peer (store + serve, no subscribe)
<code>axona_unhost</code>	<code>topic, region?</code>	Stop hosting
<code>axona_status</code>	—	Peer <code>nodeId</code> + Author ID, mesh health, per-watch + hosted lists
<code>axona_subscribe</code>	<code>topic, region?, seconds? (1–120), since?</code>	One-shot listen window (back-compat)

Region defaults to `useast (0x89)`; subscribers must use the **same region** as the publisher. The server’s code default targets **production** (the 3.x line); a v4.11.2 MCP peer must set `RELAY_NETWORK=testnet` to join the 4.x line (the two don’t interoperate). Within a line, publishing interoperates with the live apps on that line.

Receiving — push vs. poll. Two paths cover the same arrivals: - **Push** — every message on a watched topic is emitted as an MCP *logging notification* (`sendLoggingMessage`) to the client. This is best-effort: it reaches the MCP client, but a turn-based agent loop won’t be interrupted by it. - **Long-poll** — `axona_poll(wait:true, timeoutSec)` blocks **server-side** until a message arrives (or the timeout), returning at push latency. This is the reliable pull side. The standing watch buffers anything that lands between polls, so nothing is lost in the gaps.

The agent becomes a **live participant** by pairing `axona_watch` with a `/loop` that calls `axona_poll(wait:true)` each iteration: watch once, then poll-act-repeat. A message that asks for work is a request to execute and reply; **approval for sensitive or irreversible actions stays on the authenticated session, never the pub/sub channel** (a signed message proves the author key, but the topic is still an external data channel — see the security boundary below).

Register it as a project-scoped MCP server with a `.mcp.json` at your repo root:

```
{
  "mcpServers": {
    "axona": { "command": "node", "args": ["/abs/path/to/axona-relay/src/mcp.js"] }
  }
}
```

Point it at testnet (or a specific bridge) with an `env` block — also `MCP_REGION` (default `useast`), `MCP_AUTHOR_PATH` (identity file), `MCP_BUFFER_CAP` (per-watch ring, default 1000):

```
{
  "mcpServers": {
    "axona": {
      "command": "node",
      "args": ["/abs/path/to/axona-relay/src/mcp.js"],
      "env": { "RELAY_NETWORK": "testnet" }
    }
  }
}
```

Security boundary. The MCP peer is a full publisher/subscriber/host with a durable identity — treat it as a real network participant. Requests and reversible work can flow over a topic, but the agent must keep the approval gate for destructive, irreversible, or outward-facing actions on its authenticated session, not on an Axona message: a signed envelope authenticates *who*, but an open-topic message is still untrusted external input.

4c. Desktop app (`axona-relay/desktop`)

An **Electron** GUI wrapper around the relay — the same hosting + metrics node, with a window instead of a terminal. Useful for non-operators who want to contribute a durable node by leaving an app open. Because it embeds a relay, a running desktop app adds one node to your local fleet (and respawns it on crash).

5. Directory & federation

A single bridge is a single point of failure. Axona avoids that with two cooperating behaviors, both built into the bridge's embedded peer:

- **Directory.** Each bridge with `BRIDGE_DIRECTORY=on` publishes a signed entry (`{url, lat, lng, label, ver, ts}`) to the well-known open topic `axona:bridge-directory` (pinned to the `useast` region so everyone derives the same Topic ID) and `host()`s that topic so the entry survives. At launch a client collects the directory, ranks candidates (configured roots → known-good by reputation → fresh from the directory), and **fails over** to a saved alternate if its primary is unreachable.
- **Federation.** A bridge with the directory on also **uplinks** — an outbound `webTransport` connection *into* a sibling bridge — so the two bridges share one connectome. A client connected

to **either** bridge can then discover and reach peers on **both**. In the live network, `bridge-west` uplinks to `bridge.axona.net`; the east bridge stays uplink-less as the seed.

Verify federation against a **warm** topic (both bridges host + republish `axona:bridge-directory`), never a cold fresh topic — a cold topic has no roots yet and reads as a false zero.

6. The PoW collector

`pow-collector.js` (`npm run collect`) is a small standing service that subscribes to the proof-of-work benchmark **results** topic and archives each published result for later analysis. It's optional infrastructure for the PoW research line, not part of the core pub/sub path. (It can wedge silently — alive but not receiving — so a restart with `since:'all'` backfills the retained backlog.)

7. Choosing what to run

You want to...	Run
Let strangers connect to your app's network	A bridge (or use the live one for your line — <code>testnet.axona.net</code> for 4.x, <code>bridge.axona.net</code> for 3.x)
Keep a topic's messages alive when no author is online	A relay that <code>host()</code> s it
Publish live subscriber counts for a topic	A relay (metrics on — the default)
Publish/subscribe from a shell, cron, or CI	The CLI (<code>axona-cli</code>)
Give an AI agent native Axona tools	The MCP server (<code>axona-mcp</code>)
Contribute a node from a desktop, no terminal	The desktop app
Survive a bridge outage	Two federated bridges + the directory

Most application developers run **none** of these in development — they point the library at the live `bridge.axona.net` and let the existing relay fleet host their topics. You reach for self-hosted services when you want an isolated network, guaranteed topic durability, or an agent/automation integration.

8. Are there analogous systems?

If you know other realtime/decentralized stacks, the Axona services map onto patterns you've already seen:

Axona service	Closest analogue elsewhere
Bridge (signaling + TURN)	WebRTC STUN/TURN servers; libp2p circuit-relay v2 / rendezvous
Relay (durable topic host)	An MQTT broker / NATS server with retained messages — but decentralized and unprivileged
Bridge directory	DNS seeds / bootstrap node lists (Bitcoin, IPFS)
MCP server	Any other MCP integration (GitHub, Slack) — a thin agent-facing adapter over the library

The difference in every row is the same: in those systems the server is an *authority* (it owns the namespace, the messages, or the membership). In Axona the service is a *volunteer* — a peer that took on a role and can be replaced by any other peer that takes on the same role.

Where to go next

- **Programmer Guide** — build the peer that talks to these services.
- **API Reference** — `host()` / `unhost()`, `metricTopic()`, and the rest of the public surface.
- **axona-bridge/deploy/INSTALL.md** — provision a bridge (installer, Docker, or manual).
- **axona-relay/README.md** — the relay's full configuration + dashboard reference.

Found a gap in this guide? Open an issue at <https://github.com/axona-net/axona-docs>.